CSC 4OS02 - Systèmes parallèles et distribués

Projet : Optimisation Par Colonie De Fourmi D'un Chemin Sur Un Paysage Fractal

Guilherme dos Santos Wisniewski

Letícia Maria Resende

Thiago Rosa Carvalho

Mars 2026

1 Infos importants

- `src/ant_simu_cli.cpp` : exécutable headless
- `src/pheronome.hpp` : mise à jour thread-safe des phéromones
- `src/Makefile` : compilation
- Code écrit en C++17 et compilé avec `-O3 -march=native`.
- Github Link

2 Objectif

Ce document décrit la réalisation du projet d'optimisation par colonies de fourmis (ACO) sur un paysage fractal, en suivant les consignes du `Readme.md` (création d'environnement fractal, simulation de fourmis, mesures de temps, vectorisation et parallélisation).

Le code a été adapté pour fonctionner sans dépendance à SDL2 (exécution sans interface graphique) et pour mesurer les temps passés dans les différentes phases de la simulation.

3 Modifications principales apportées

3.1 Exécution headless (sans SDL)

- Le projet original dépendait de la bibliothèque SDL2 pour l'affichage (fichiers `window.cpp`, `renderer.cpp`, `ant_simu.cpp`).
- Pour garantir une exécution sur toutes les machines (en particulier dans cet environnement où SDL2 n'était pas installée), j'ai ajouté un exécutable **headless** : `ant_simu_cli.exe`.
- Le Makefile a été modifié pour construire `ant_simu_cli.exe` par défaut et pour laisser `ant_simu.exe` en tant que cible optionnelle (`make sdl`).

3.2 Mesure des temps

L'exécutable `ant_simu_cli.exe` mesure :

- le temps total de la simulation,
- le temps passé dans le déplacement des fourmis,
- le temps passé dans l'évaporation + mise à jour des phéromones,
- le temps moyen par itération.

3.3 Parallélisation OpenMP

- La boucle principale mettant à jour les fourmis est parallélisée avec OpenMP (`#pragma omp parallel`).
- La mise à jour des phéromones se fait sans contention en utilisant un *buffer par thread* : chaque thread accumule localement ses contributions dans une grille locale, puis ces grilles sont fusionnées par une réduction `max` à la fin de l'itération.
- La phase d'évaporation des phéromones est également parallélisée (chaque cellule est traitée indépendamment).

4 Compilation

Dans le dossier `projet/src`, exécuter :

```
make
```

Pour compiler également l'exécutable SDL (si SDL2 est disponible) :

```
make sdl
```

5 Utilisation de l'exécutable headless

L'exécutable `ant_simu_cli.exe` prend en entrée plusieurs paramètres :

- `-i <iter>` : nombre d'itérations (par défaut 1000)
- `-a <ants>` : nombre de fourmis (par défaut 5000)
- `-t <threads>` : nombre de threads OpenMP (par défaut 1)
- `-s <seed>` : graine aléatoire (par défaut 2026)
- `-e <eps>` : coefficient d'exploration (epsilon) (par défaut 0.8)
- `-A <alpha>` : coefficient de bruit pour la mise à jour des phéromones (par défaut 0.7)
- `-b <beta>` : coefficient d'évaporation (par défaut 0.999)
- `-g <log2dim>` : dimension de la grille = 2^{log2dim} (par défaut 8)
- `-c` : affiche les résultats au format CSV
- `-v` : mode verbeux

Exemple :

```
./ant_simu_cli.exe -i 200 -a 2000 -t 4
```

6 Résultats des mesures

6.1 Impact de l'optimisation séquentielle (AoS vs SoA)

Avant d'analyser le parallélisme, nous comparons le code original (séquentiel, basé sur des objets) avec notre version optimisée (CLI) exécutée sur un seul thread.

Mesure	Code Séquentiel (Original)	Code CLI (1 Thread)	Gain / Speedup
Temps Total (s)	61.97	23.86	**2.60**×
Temps Déplacement (s)	26.85	19.13	**1.40**×
Temps Phéromones (s)	1.10	1.32	0.83×
Fourmis collectées	2328	4666	**2.00**×

Analyse du gain séquentiel :

— **Efficacité Cache :** Le passage à une structure *Structure of Arrays* (SoA) permet d'obtenir un speedup global de **2.6**× sans même utiliser plusieurs cœurs. Cela démontre que la réorganisation de la mémoire est l'étape la plus critique.
— **Efficacité Algorithmique :** On note que la version CLI collecte deux fois plus de nourriture (4 666 vs 2 328). Cela suggère que la nouvelle implémentation de la mise à jour des phéromones (probablement via une réduction `max`) favorise une convergence plus rapide vers les chemins optimaux.

6.2 Scalabilité forte (strong scaling)

Expériences réalisées avec la version optimisée CLI : `-i 2000 -a 20000 -g 8`.

Threads	Total (s)	Déplacement (s)	Phéromones (s)	Accélération	Efficacité
1	23.86	19.13	1.32	1.00×	100%
2	14.31	10.19	0.79	1.67×	83%
4	12.10	7.06	0.63	1.97×	49%
8	12.48	4.84	0.53	1.91×	24%

Observation : Si l'on compare le code parallèle à 8 threads (12.48s) au code séquentiel original (61.97s), le speedup total est de **4.96**×.

6.3 Scalabilité faible (weak scaling)

Dans cette expérience, le nombre de fourmis augmente proportionnellement au nombre de threads (5 000 fourmis par cœur). L'objectif est d'évaluer la capacité du système à maintenir un temps d'exécution constant face à une charge croissante.

Expériences réalisées avec : `-i 2000 -g 8`.

Threads	Fourmis	Total (s)	Déplacement (s)	Phéromones (s)	Efficacité
1	5 000	9.66	4.97	1.27	100%
2	10 000	9.70	5.55	0.81	99.6%
4	20 000	12.27	7.14	0.65	78.7%
8	40 000	15.61	8.03	0.54	61.9%

Observation :

— Entre 1 et 2 threads, l'efficacité est presque parfaite ($> 99\%$), le temps total reste stable malgré le doublement de la charge.
— À partir de 4 threads, on observe une dégradation du temps de déplacement ($4.97s \rightarrow 8.03s$ pour une charge par thread théoriquement identique). Cela indique une saturation de la bande passante mémoire ou un overhead de gestion des threads OpenMP.
— Fait intéressant : le temps consacré aux phéromones diminue alors que la charge augmente, ce qui montre que la parallélisation de la mise à jour de la grille est très efficace.

6.4 Impact des paramètres de l'algorithme

Cette série d'expériences (réalisée avec 2 000 itérations, 5 000 fourmis et 4 threads) permet d'isoler l'influence des hyperparamètres sur l'efficacité de la recherche de chemin et sur le temps de calcul.

Paramètre	Valeur	Total (s)	Déplacement (s)	Collectées	1ère Nourriture
ϵ (Exploration)	0.1	7.23	2.28	13	1493
	0.5	7.03	2.21	13	1493
	0.9	7.05	2.17	13	1493
α (Bruit)	0.1	6.73	2.11	11	1460
	0.5	9.20	2.84	11	1471
	0.9	9.90	3.08	12	1490
β (Évaporation)	0.500	7.50	2.25	2	1885
	0.800	7.13	2.22	10	1518
	0.950	7.02	2.19	11	1490
	0.999	7.29	2.22	13	1493

Analyse des observations :

- **Coefficient d'exploration (ϵ) :** À ce niveau de densité de fourmis, ϵ n'a quasiment aucun impact sur le temps de calcul ni sur le succès de la colonie. Cela suggère que le comportement est dominé par le paysage fractal lui-même ou que le seuil de basculement vers l'exploitation n'est pas atteint.
- **Coefficient de bruit (α) :** On observe une corrélation surprenante : augmenter α semble augmenter légèrement le temps de déplacement et le temps total. Cela peut s'expliquer par une modification de la localité mémoire (accès aux phéromones plus erratiques) ralentissant les caches processeurs.
- **Évaporation (β) :** C'est le paramètre le plus influent sur la convergence. Avec une évaporation forte ($\beta = 0.5$), la mémoire collective de la colonie est trop courte : seules 2 fourmis parviennent à ramener de la nourriture. À l'inverse, une persistance élevée ($\beta = 0.999$) permet de stabiliser les pistes de phéromones, augmentant le succès de collecte de 550% par rapport à $\beta = 0.5$.

6.5 Impact de la taille du problème

log2dim	Grille	Total (s)	Temps phéromones (s)	Fourmis collectées
7	128×128	2.52	0.32	18450
8	256×256	12.10	0.63	4666
9	512×512	48.45	2.15	1240

Observation : le coût de calcul augmente fortement avec la taille de la grille, dominé par la mise à jour des phéromones.

7 Stratégies de parallélisation avancées

7.1 Vectorisation et réorganisation mémoire (SoA)

Une optimisation consiste à stocker les états des fourmis sous forme de *structure of arrays* :

- `std::vector<int> ant_x`
- `std::vector<int> ant_y`

— `std::vector<char> ant_loaded`
— `std::vector<std::size_t> ant_seed`

Cela améliore la localité mémoire et facilite les optimisations SIMD.

7.2 Parallélisation mémoire partagée (OpenMP)

La boucle sur les fourmis est parallélisée, et la parallélisation peut être étendue aux phases :

— évaporation des phéromones
— mise à jour de la grille

7.3 Parallélisation distribuée (MPI)

Deux approches possibles :

— **Carte complète + partition des fourmis** : chaque processus possède une copie de la carte et les phéromones sont réduites par `max`.
— **Décomposition de domaine** : chaque processus gère une sous-partie de la grille et les échanges se font aux frontières.

8 Comparaison entre `ant_simu.cpp` et `ant_simu_cli.cpp`

Cette section détaille les différences structurelles et algorithmiques entre la version originale (graphique) et la version optimisée pour le calcul haute performance (CLI).

8.1 Organisation générale de la simulation

L'implémentation originale (`ant_simu.cpp`) est conçue autour de la fonction `advance_time(...)` qui encapsule la logique principale de la simulation. Cette fonction parcourt l'ensemble des fourmis et appelle la méthode `advance(...)` de chaque objet `ant`. La logique de déplacement, de décision et d'interaction avec les phéromones est donc principalement contenue dans la classe `ant`.

Dans la version CLI (`ant_simu_cli.cpp`), la logique de simulation n'est plus encapsulée dans la classe `ant`. Le comportement des fourmis est directement implémenté dans la boucle principale de la simulation. Cette restructuration permet un contrôle plus fin du flux d'exécution et facilite l'introduction de parallélisme.

8.2 Parallélisation et Structure des données

La différence la plus importante concerne l'introduction du parallélisme et la gestion de la mémoire.

— **Parallélisme :** Dans la version originale, la simulation est entièrement séquentielle. Dans la version CLI, la boucle principale est parallélisée à l'aide d'OpenMP (`#pragma omp parallel for`). Des mécanismes de synchronisation (opérations atomiques et buffers locaux) sont introduits pour éviter les conditions de concurrence.
— **AoS vs SoA :**

— *Original (AoS) :* Les fourmis sont des objets `std::vector<ant>`. Chaque objet contient son propre état (position, graine aléatoire).
— *CLI (SoA) :* Les informations sont stockées dans plusieurs tableaux séparés (`ant_x`, `ant_y`, etc.). Cette organisation en **Structure of Arrays** améliore la localité mémoire et favorise la vectorisation SIMD.

8.3 Instrumentation et Paramétrisation

— **Mesure de performance :** La version CLI introduit un système basé sur `std::chrono` pour décomposer le temps d'exécution (déplacement vs phéromones), outil absent de la version originale.
— **Configuration :** Dans la version originale, les paramètres sont "hard-codés". La version CLI permet de configurer la simulation via la ligne de commande (`-i`, `-a`, `-t`, etc.), facilitant l'automatisation des tests de performance.

8.4 Contrôle et Dépendances

— **Boucle de contrôle :** La version graphique est synchronisée sur le taux de rafraîchissement de l'écran (via SDL). La version CLI est pilotée par un nombre fixe d'itérations, garantissant la reproductibilité des mesures.
— **Environnement :** La version CLI supprime toute dépendance à SDL2, permettant l'exécution sur des serveurs de calcul (clusters) ou des environnements sans interface graphique.